

Assignment-13.4

Name: Sushma Purella

H. No:2303A52188

Batch:44

Task 1: Refactoring Data Transformation Logic

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Prompt:

Refactor the following Python code into a more Pythonic form using list comprehensions while keeping the same output.

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)

print(doubled)
```

Code:

```
10 #Refactored Code
11 values = [2, 4, 6, 8, 10]
12 doubled = [v * 2 for v in values]
13 print(doubled)
14 #In the refactored code, we utilize a list comprehension to create the 'doubled'
15 # list in a more concise and readable manner. This approach eliminates the need for an explicit
16 # loop and the intermediate 'append' method, resulting in cleaner and more efficient code.
17
18
19
20
```

Loading...

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-13.4.py
[4, 8, 12, 16, 20]
○ PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist>

Observation:

The AI suggested replacing the explicit loop with a list comprehension, which makes the code shorter and more readable. The refactored code performs the same transformation but in a single line

Task 2: Improving Text Processing Code

Readability

You are working on a log message generator that builds messages word by word.

Prompt:

Refactor the following Python code to make it more efficient and readable while keeping the output the same. Avoid repeated string concatenation.

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Code:

```
33
34 words = ["Refactoring", "with", "AI", "improves", "quality"]
35 message = " ".join(words)
36 print(message)
37 #In the refactored code, we use the 'join' method to concatenate the list of words into a single
38 # string with spaces in between. This approach is more efficient than repeated string
39 # concatenation and results in cleaner and more readable code. The final output remains unchanged
40 # producing the same message as the legacy code.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Refactoring with AI improves quality

Observation:

The AI suggested using the `join()` method instead of repeatedly concatenating strings inside a loop.

Using `" ".join(words)` combines all words in the list into a single string separated by spaces. This approach is more efficient, faster, and cleaner than repeated string concatenation. The final output remains the same

Task 3: Safer Access to Configuration Data

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Prompt:

Refactor the following Python code to use a safer and cleaner way to access dictionary values when a key may not exist. The output behavior should remain the same.

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
```

```
else:  
    print("Default timeout used")
```

Code:

```
56 #Ensure the behavior remains the same for missing keys.  
57 config = {"host": "localhost", "port": 8080}  
58 timeout = config.get("timeout", "Default timeout used")  
59 print(timeout)  
60
```

A screenshot of a code editor with a dark theme. The editor shows a Python script with five lines of code. Line 56 is a comment: "#Ensure the behavior remains the same for missing keys." Line 57 defines a dictionary: "config = {'host': 'localhost', 'port': 8080}". Line 58 uses the .get() method: "timeout = config.get('timeout', 'Default timeout used')". Line 59 prints the value of 'timeout': "print(timeout)". Line 60 is an empty line. Below the code editor, there is a panel with tabs for "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL", and "PORTS". The "TERMINAL" tab is selected, showing the command prompt: "PS C:\users\susnm\OneDrive\Attachments\Desktop\AI-Assist> & C:/users/susnm/AppData/Local/Programs/Python/Python313/py". Below the command prompt, the output "Default timeout used" is displayed.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\users\susnm\OneDrive\Attachments\Desktop\AI-Assist> & C:/users/susnm/AppData/Local/Programs/Python/Python313/py
Default timeout used

Observation:

The AI suggested using the `get()` method of dictionaries instead of manually checking if a key exists. `config.get("timeout", "Default timeout used")` safely retrieves the value of "timeout" if it exists; otherwise it returns the default message. This makes the code shorter, cleaner, and safer while keeping the same behavior when the key is missing.

Task 4: Refactoring Conditional Logic for Scalability

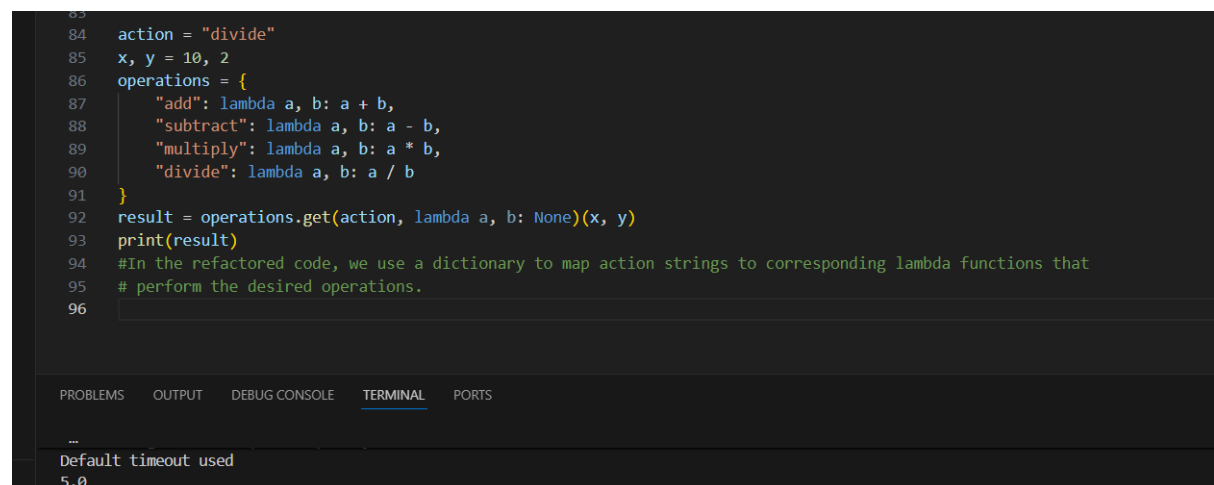
You are enhancing a utility module that performs different operations based on user input.

Prompt:

Refactor the following Python code to remove multiple if–elif conditions and use a cleaner, more scalable approach such as mapping techniques while keeping the same functionality.

```
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Code:



```
83
84 action = "divide"
85 x, y = 10, 2
86 operations = {
87     "add": lambda a, b: a + b,
88     "subtract": lambda a, b: a - b,
89     "multiply": lambda a, b: a * b,
90     "divide": lambda a, b: a / b
91 }
92 result = operations.get(action, lambda a, b: None)(x, y)
93 print(result)
94 #In the refactored code, we use a dictionary to map action strings to corresponding lambda functions that
95 # perform the desired operations.
96
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

...
Default timeout used
5.0

Observation:

The AI suggested replacing multiple if–elif conditions with a dictionary mapping technique.

Each operation name is mapped to a corresponding lambda function that performs the calculation. The `get()` method safely retrieves the

required operation, and a default function returns None if the action is not found. This approach makes the code cleaner, easier to maintain, and more scalable when adding new operations.

Task 5: Simplifying Search Logic in Collections

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Prompt:

Refactor the following Python code to make the search logic more concise and readable. Avoid using an explicit loop if possible and maintain the same output behavior.

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Code:

```
119 inventory = ["pen", "notebook", "eraser", "marker"]
120 item_to_find = "eraser"
121 if item_to_find in inventory:
122     print("Item Available")
123 else:
124     print("Item Not Available")
125 #In the refactored code, we utilize the 'in' keyword to check for the presence of the item in the inventory list.
126 #This approach is more concise and readable compared to the explicit loop used in the legacy code,
127 # while still maintaining the same output behavior.
128
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

...
Item Available

Observation:

The AI suggested replacing the explicit loop with the `in` membership operator. This operator directly checks whether an item exists in a list, eliminating the need for a manual loop and flag variable. The refactored code is shorter, more readable, and more Pythonic, while producing the same output as the legacy code.